

Sistemas de Información

Práctica 4

Cardiel, Enrique
874417@unizar.es

Sierra, Víctor
869244@unizar.es

Blanchard, Daniel
870373@unizar.es

Noviembre 2025

Contents

1	Estructura del sistema de información	3
1.1	Esquema de la aplicación	3
1.2	Documentación del código	3
2	Funcionalidades principales respecto al primer informe	3
2.1	Perfiles	3
2.2	Proceso de compra	4
2.3	Funcionalidades a completar	4
3	Seguridad y mantenimiento	5
4	Herramientas utilizadas	5
5	Metodología	5
6	Dificultades Encontradas	6

1 Estructura del sistema de información

1.1 Esquema de la aplicación

La estructura de la aplicación está dividida según la capa de persistencia, lógica de negocio y presentación (view layer).

- **Capa de persistencia en `project/db_utils`:** Se encuentran todas las clases DAO y VAO implementadas en python.
- **`project/src`:** Aquí se encuentran el resto de ficheros *source* del código. Está dividida en capa de lógica de negocio y view layer.
- **`project/src/Backend`:** Capa de lógica de negocio. Los ficheros principales son:
 - **`application.py`** : Es el fichero principal de la API FastAPI. En él se definen todos los endpoints de todas las funcionalidades posibles de la aplicación.
 - **`model.py`** : Define los modelos Pydantic usados para la validación automática de los datos que recibe FastAPI.
 - **`services.py`** : En él se comprueban elementos de la lógica de negocio como las contraseñas (generando también hashes de las mismas), así como el servicio de envío del **correo de bienvenida** o de confirmación de pedido.
- **`project/src/Frontend`:** *view layer*. En este caso se sigue la distribución típica en React, y se distingue entre componentes reutilizables (components/), páginas independientes (pages/) y los estilos en formato .css (css/)

El flujo típico de la información en el sistema de información creado sería:

Frontend → application.py (endpoint) → services.py (lógica) → db_utils (base de datos)

1.2 Documentación del código

Respecto a la documentación, la herramienta FastAPI facilita la visualización de la documentación del Frontend, y se puede acceder a ella mediante la URL <http://127.0.0.1:8000/docs> que se crea al iniciar la API.

2 Funcionalidades principales respecto al primer informe

2.1 Perfiles

Cuando estábamos comenzando con el proyecto, teníamos una ligera idea de qué perfiles queríamos en Yami pero nada seguro. Sin embargo, casi dos meses más tarde, ya los tenemos bien definidos.

Por un lado está el **Cliente**, la persona que pide comida a domicilio a través de nuestra

página web. Puede observar todos los restaurantes que hay y los platos que ofrecen, consultar sus descripciones y añadirlos a un carrito en el que finalizará su compra.

Por otro lado están los **Restaurantes**, que se encargan de ofrecer una serie de platos a los clientes dados de alta en Yami. Cuando reciben un pedido, se encargan de actualizar el estado de este, para tener informado al cliente.

Finalmente se encuentra el **Administrador**, que es el que se encarga de gestionar la página web. Entre sus funciones se encuentran modificar datos de restaurantes o clientes e incluso eliminar platos de restaurantes o restaurantes que no cumplan con la normativa.

Notar que hemos terminado descartando el perfil de **Repartidor**, puesto que debido al tiempo del que disponemos se nos hacía imposible incluir este rol en Yami.

2.2 Proceso de compra

El proceso de compra que atraviesa un cliente es el siguiente.

Primero, debe iniciar sesión con sus claves pertinentes (correo y contraseña). Una vez iniciada, se le redirige a la página donde se muestran todos los restaurantes dados de alta en Yami. También dispone de filtros que criban los locales según el tipo de comida ofrecida.

Al clicar en un restaurante específico, se muestran todos los platos de ese restaurante junto a otros dos filtros, uno de los alérgenos y otro del tipo de plato que quiere (Entrante, Principal y Postre). Si ve un plato que le resulte atractivo, puede ir a la página del plato en la que se muestra un botón de añadir al carrito.

Es importante destacar que si sales de la página del restaurante con elementos en el carrito, estos son eliminados puesto que en un mismo pedido no se pueden encargar platos de distintos sitios.

2.3 Funcionalidades a completar

Dado que los tres integrantes del grupo somos del Programa Conjunto de Matemáticas e Ingeniería Informática, no nos ha dado tiempo a avanzar tanto como nos hubiera gustado.

Aún quedan por implementar las funcionalidades de Restaurante y del Administrador, pero muchas pantallas son prácticamente recicladas de las diseñadas para el Cliente (sobre todo las que enseñamos una lista de elementos, en este caso Restaurantes y Platos). También sería necesaria la parte del perfil de usuario, tanto de restaurante como de cliente, donde poder editar sus datos o borrar el perfil si es necesario.

Por esta razón, no creemos que nos lleve mucho tiempo una vez que nos pongamos, puesto que ya no nos cuesta tanto como al principio programar en HTML.

3 Seguridad y mantenimiento

Como tenemos nuestra base de datos en Supabase, esta misma página previene las SQL Injections, por lo que no nos hemos tenido que preocupar por este aspecto.

En cuanto al mantenimiento de la aplicación, son los administradores los que se encargan de comprobar que todo cumple con la normativa correspondiente, pudiendo ejercer las medidas necesarias en caso contrario.

Si continuáramos con el proyecto, una de las posibilidades sería terminar implementando el rol de **Repartidor**, a la vez que confirmar la seguridad de nuestra base de datos.

4 Herramientas utilizadas

Backend

Para el desarrollo de Backend como se ha ido comentando previamente hemos hecho uso de Python y en concreto de *FastAPI*. La elección de *FastAPI* fue tomada por varios motivos, el primero de ellos y fundamental era su sintaxis fácil que nos permitió entender la herramienta desde el primer instante facilitando el desarrollo del backend. Otro punto positivo de FastAPI es su generación automática de documentación que facilitó la integración del Backend con el Frontend.

A su vez también hemos creado una cuenta de correo de google *alerts.yami.app@gmail.com* a través de la cual podemos enviar correos a nuestros clientes, para darles la bienvenida a la app o para informarles del estado de sus pedidos.

Frontend

Para el desarrollo del Frontend hemos decidido usar *React*. Hemos decidido usar *React* porque es una tecnología puntera en el desarrollo de frontend, por lo tanto nos parecía de gran utilidad aprender esta tecnología. El desarrollo del frontend cuenta con ficheros de *JavaScript* donde implementamos la lógica de las páginas así como el contenido HTML y también ficheros de CSS donde modelamos la apariencia de la página tratando de ser lo más fiel posible a nuestros primeros diseños en *Figma*.

5 Metodología

En esta cuarta práctica hemos seguido con la metodología de las tres prácticas anteriores. En esta práctica hemos rotado el "líder", en este caso el encargado de liderar la práctica ha sido **Daniel Blanchard** y ha sido quien ha realizado la entrega.

En concreto, la división de tareas ha sido la siguiente:

- Víctor Sierra (20 horas):
 1. Elaboración del Backend (6 horas)
 2. Diseño Página de Restaurantes (2,5 horas)

3. Diseño Página de Plato (2,5 horas)
 4. Diseño de Filtros Funcionales (4,5 horas)
 5. Rellenar Base de Datos (2,5 horas)
 6. Memoria (2 horas)
- Daniel Blanchard (26 horas):
7. Elaboración del Backend (18 horas)
 8. Revision del Modelo Entidad-Relación y modelo relacional (2 hora)
 9. Revision de DAO y VO (2 horas)
 10. Contribucion al Frontend (2 horas)
 11. Memoria (1 horas)
 12. How To Install My Application (1 horas)
- Enrique Cardiel (16,5 horas):
12. Página Principal (Frontend) (3 horas)
 13. Diseño de la Estructura del Frontend (footer, header, etc.)(6 horas)
 14. Conexión entre Frontend y Backend (3 horas)
 15. Formulario Dar de Alta y Log In (2 horas)
 16. Revisión de funcionalidades de Backend (2 horas)
 17. Revisión de la memoria (0,5 horas)

6 Dificultades Encontradas

Las principales dificultades que hemos tenido que afrontar en esta practica se pueden resumir en dos aspectos. Dificultades tecnicas y temporales.

Pese a la larga duración de esta practica, nos ha resultado difícil poder dedicar el tiempo que nos habría gustado a esta practica debido a un horario universitario exigente que no nos ha permitido desarrollar la aplicación al nivel que nos gustaría. En las próximas semanas dispondremos de mayor tiempo para poder seguir desarrollando y puliendo Yami.

En lo que respecta a las dificultades técnicas, principalmente han sido las curvas de aprendizaje tanto en el Backend como en el Frontend. El Backend no resulto ser muy complicado debido a que *FastAPI* nos facilito la labor y pudimos elaborar un buen back-end en relativamente poco tiempo. El desarrollo del Frontend ha sido el verdadero reto de esta practica. Es la primera vez que los tres programamos en HTML, CSS y JavaScript y por lo tanto el proceso de aprendizaje ha sido mas lento de lo que nos hubiera gustado.